CIS437 - Cloud Computing

– Microservices –

From Monolithic Applications to Microservices

Instructor: Yong Zhuang

yong.zhuang@gvsu.edu

Based on the original version by Professor Erik Fredericks

Last Class

- Cloud patterns solve recurring architectural problems.
- **Pub/Sub:** decouples publishers and subscribers.
- **Geode:** distributes applications geographically.
- **API Gateway:** provides one entry point to multiple backend services.
- **Circuit Breaker:** protects systems from repeatedly calling an unhealthy dependency.
- **Choose a pattern based on the problem you need to solve.**

Last Class Practice

For each scenario, identify the most appropriate pattern or patterns and briefly explain why.



1. A mobile application needs to access Catalog, Orders, Payments, and User services without knowing each service's address.

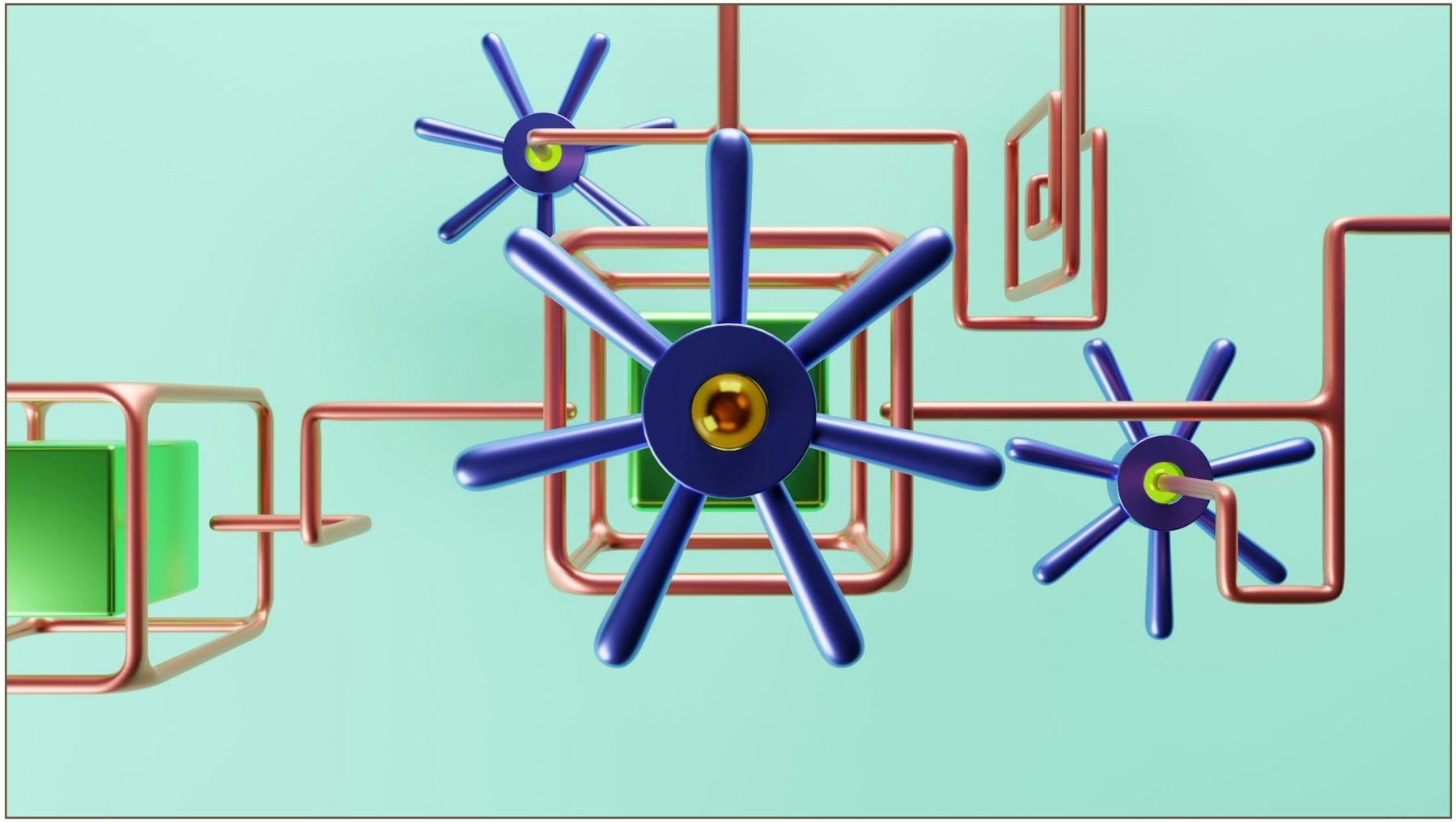
2. The Payment Service starts timing out. The Order Service keeps retrying and is consuming more resources.

3. A global shopping site runs in several geographic regions, and an order event must be processed independently by inventory, shipping, and analytics services.

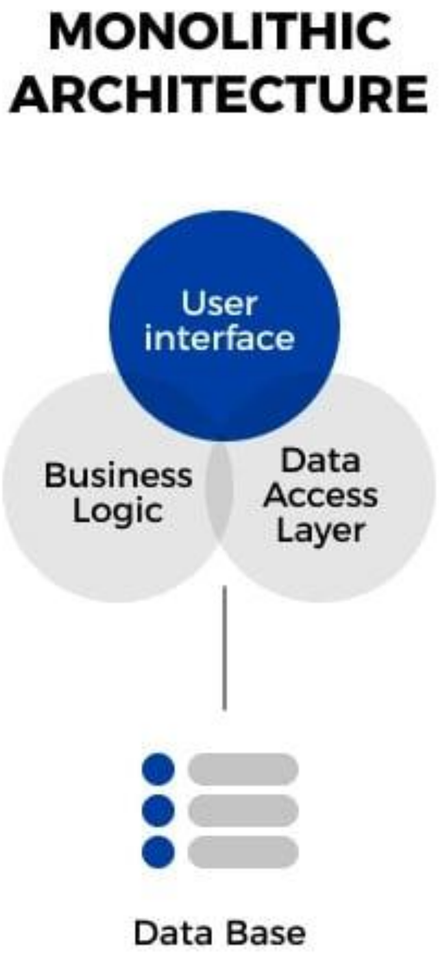
Agenda

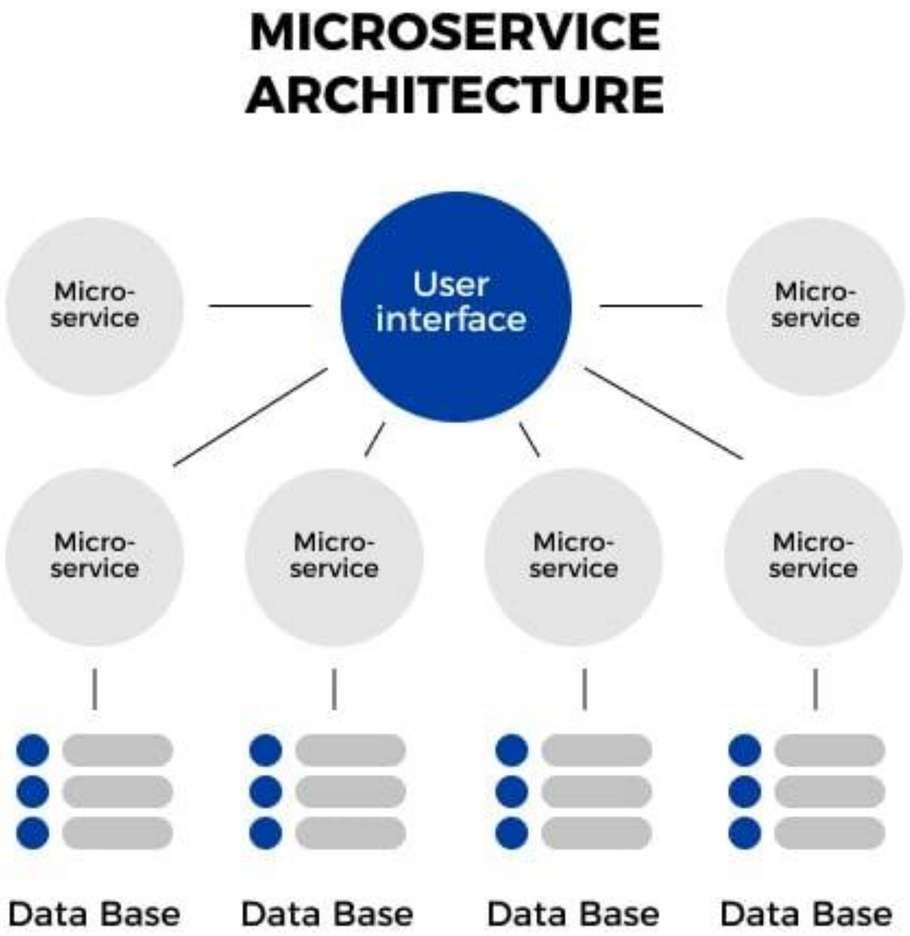
- Monolithic Applications
- Problems and Limitations of Monoliths
- Microservices Architecture
- Benefits and Challenges
- Decoupling and Service Boundaries
- When Should We Use Microservices?
- Monolith-to-Microservices Lab

What Is a Microservice?



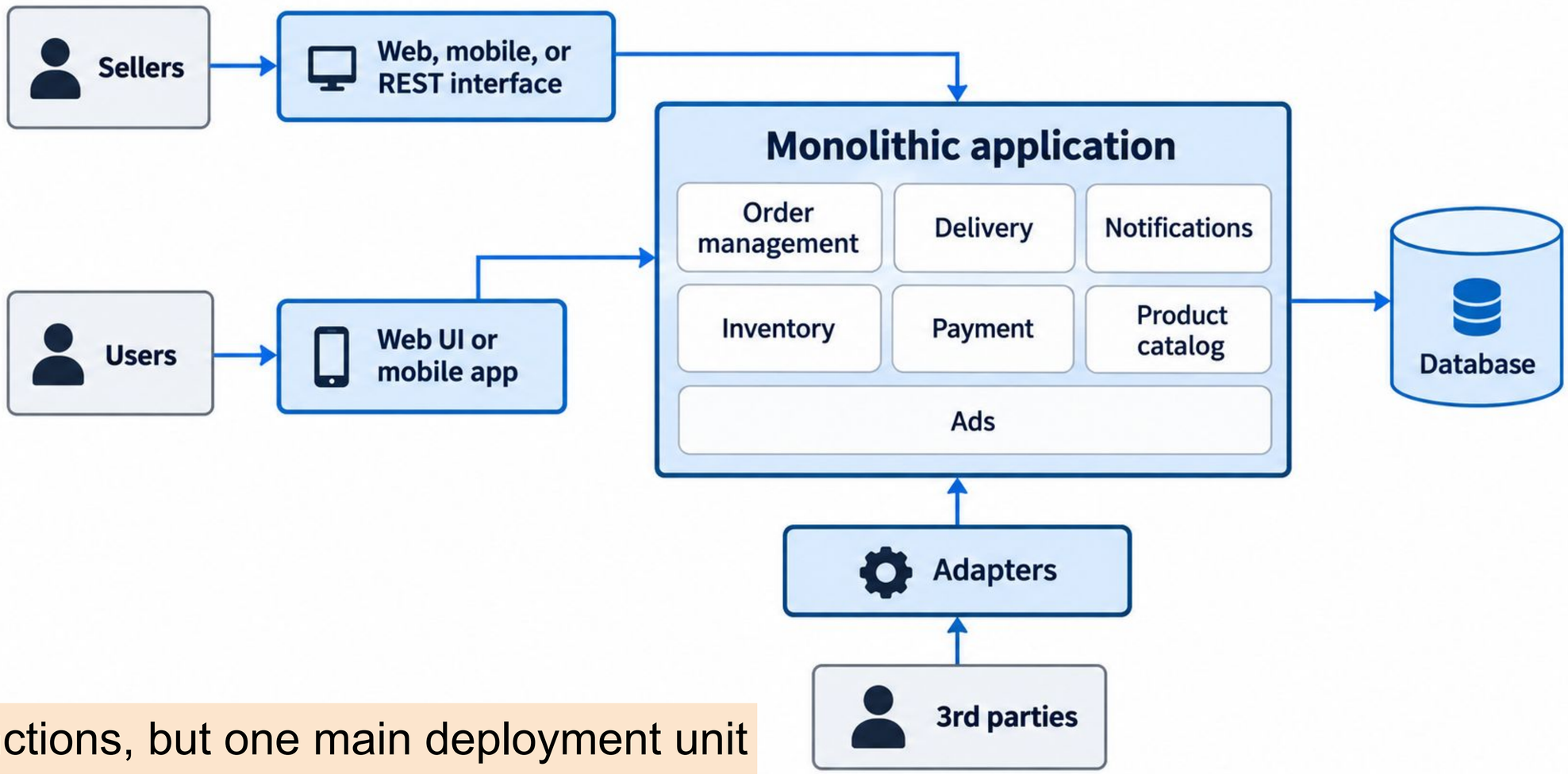
Monolithic vs. Microservices Architecture





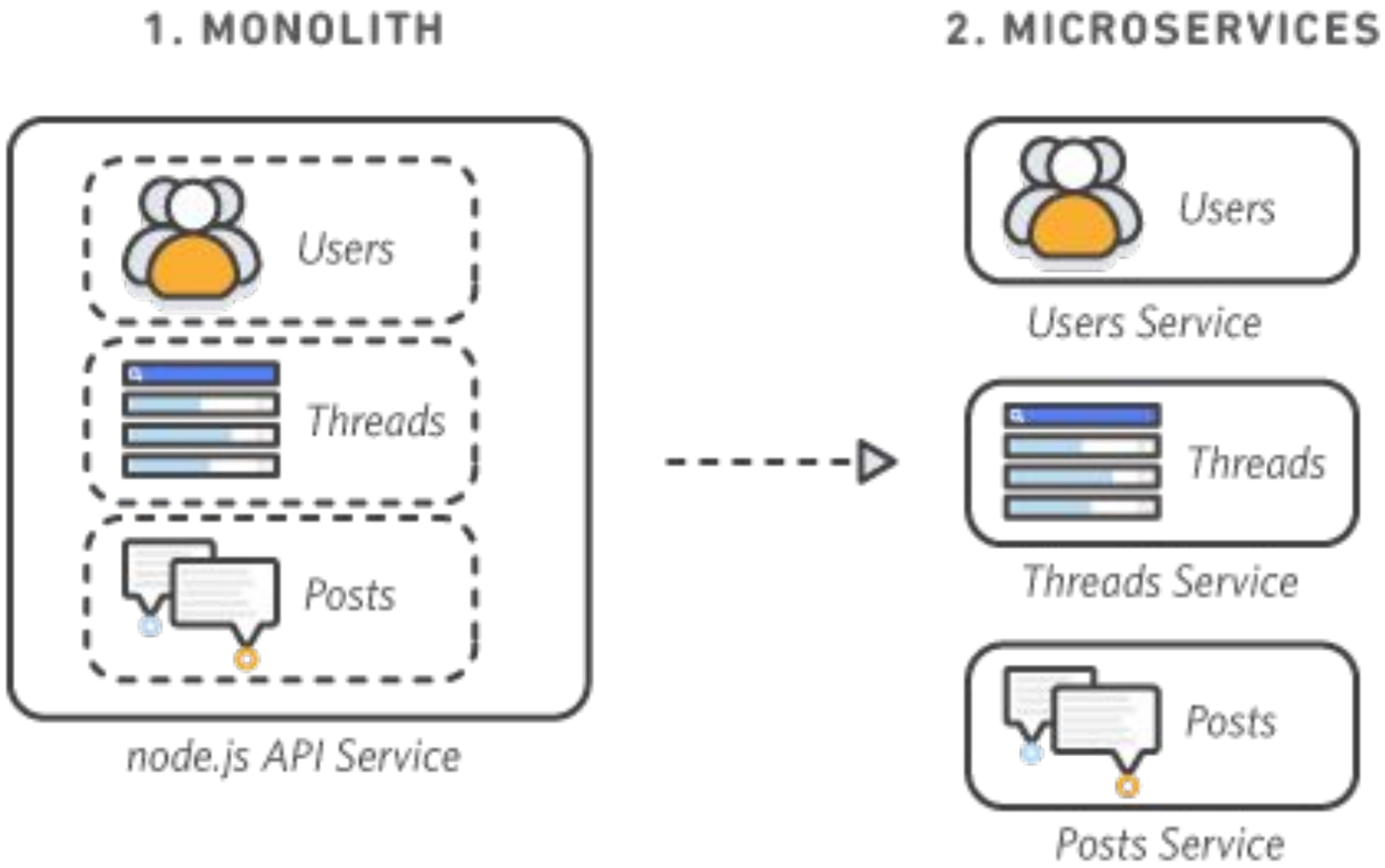
[What is Microservices Architecture?]
[Monolithic Application vs Microservices Architecture]

Example: Monolithic E-Commerce Application



Many functions, but one main deployment unit

Breaking a Monolith into Services



AWS: Microservices

What is a *monolithic* application?

- An application in which many functions are developed and deployed together as one unit
- Often described as “one big application”
- Different features may exist as modules inside the application
- Components are often tightly connected and share the same deployment lifecycle

Key idea:

A change to one part may require rebuilding and redeploying the whole application.

Monolith = many functions, one main deployment unit

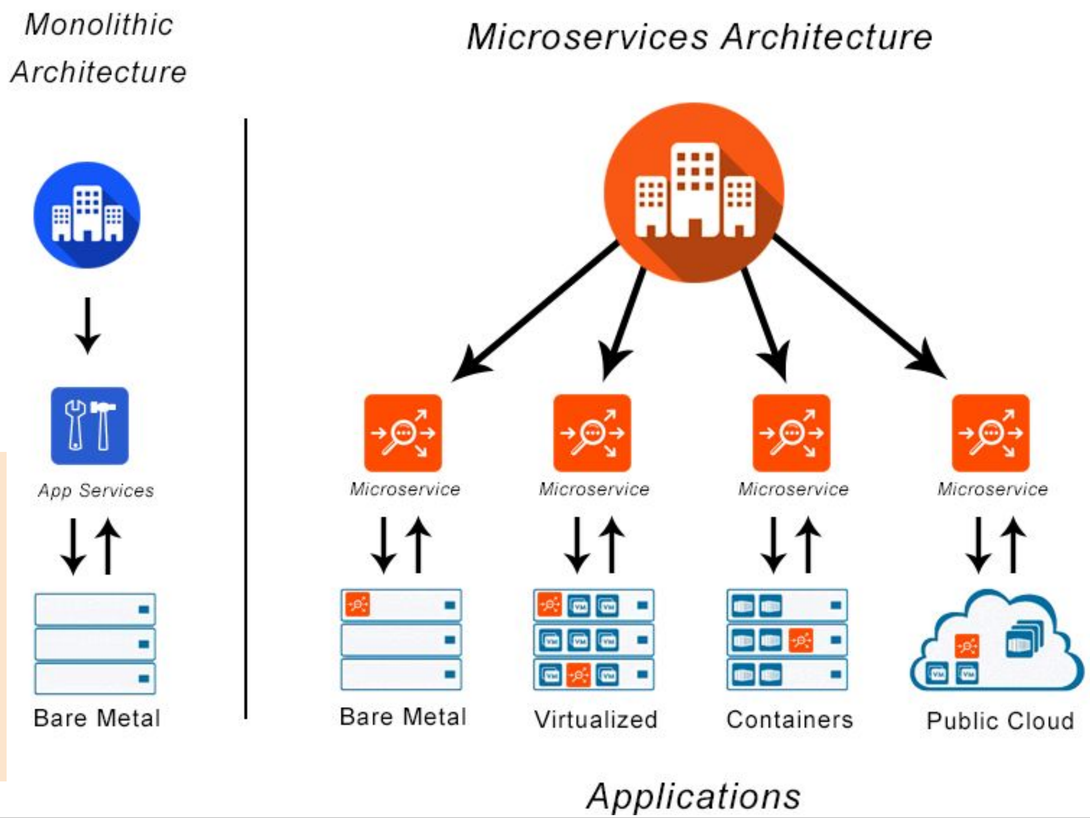
Challenges of a Growing Monolith

- **Updates can become more difficult**
  - A change to one feature may require testing and redeploying the whole application
- **Independent scaling can be difficult or inefficient**
  - If only one feature needs more capacity, you may still need to scale the whole application
- **Cloud migration may require refactoring**
  - Especially for older applications that depend on local infrastructure or tightly coupled components



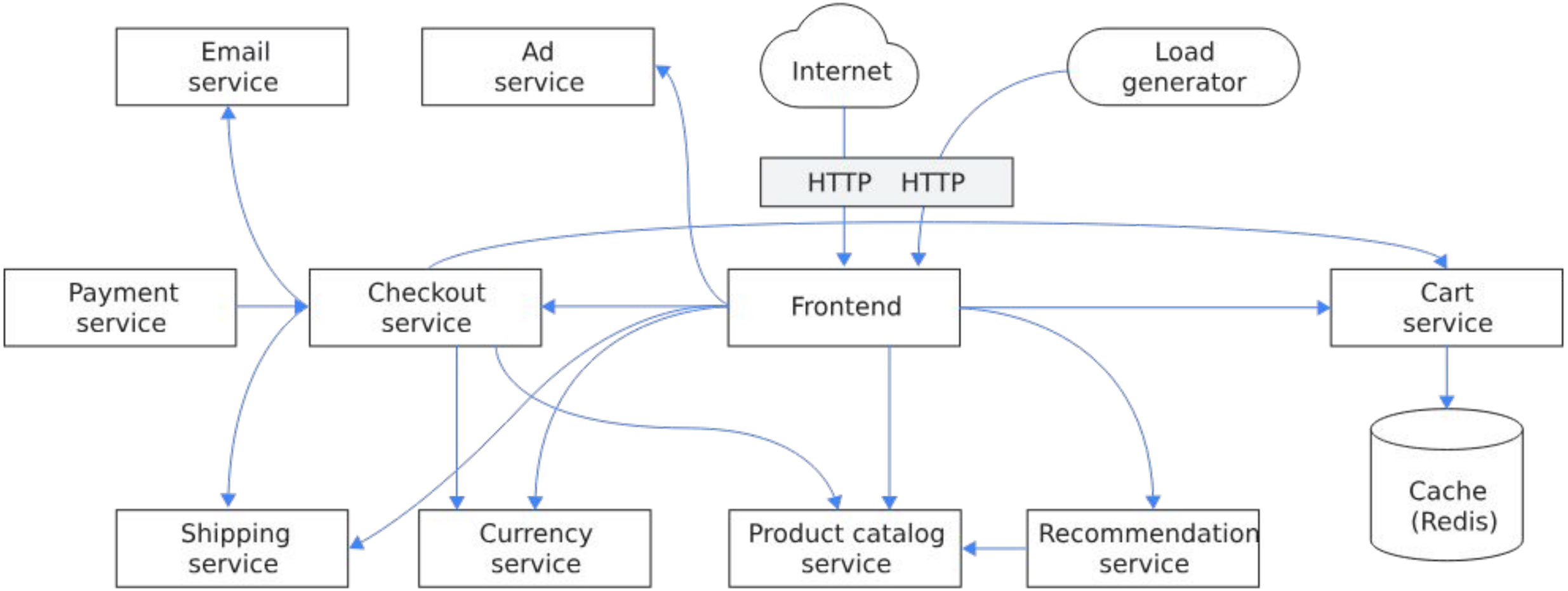
Microservices Architecture

- Application composed of multiple focused services
- Each service has a specific responsibility or business capability
- Services communicate through well-defined APIs or messages
- Services are loosely coupled
- Services can be deployed and scaled independently



- An application composed of independently deployable services organized around specific responsibilities or business capabilities.
- Loosely coupled does not mean disconnected. The services still interact, but through well-defined interfaces/APIs.

Monolith to microservices



One application can become a network of many cooperating services

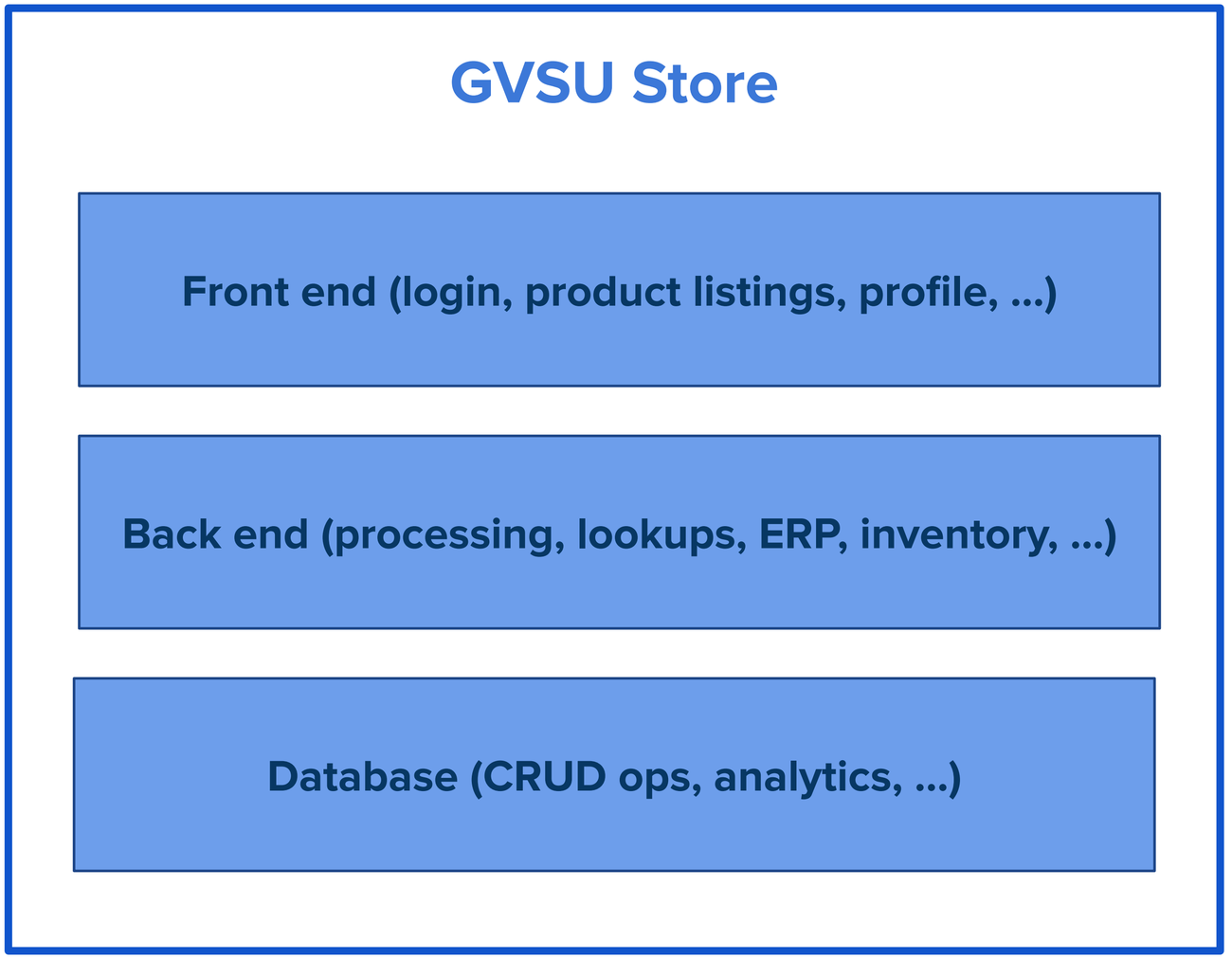
Example: GVSU Store Monolith

Start with a large monolithic application

- For example, an online store

Many application functions are developed and deployed together as one large application.

How do we go about turning it into a collection of microservices?



Benefits of Microservices

- **Separation of concerns**
  - Each service focuses on a specific responsibility
- **Independent development and deployment**
  - Different teams can work on different services
  - Works well with CI/CD pipelines
- **Independent scaling**
  - Scale only the services that need more capacity
- **Smaller, isolated updates**
  - Changes to one service do not necessarily require redeploying the whole application

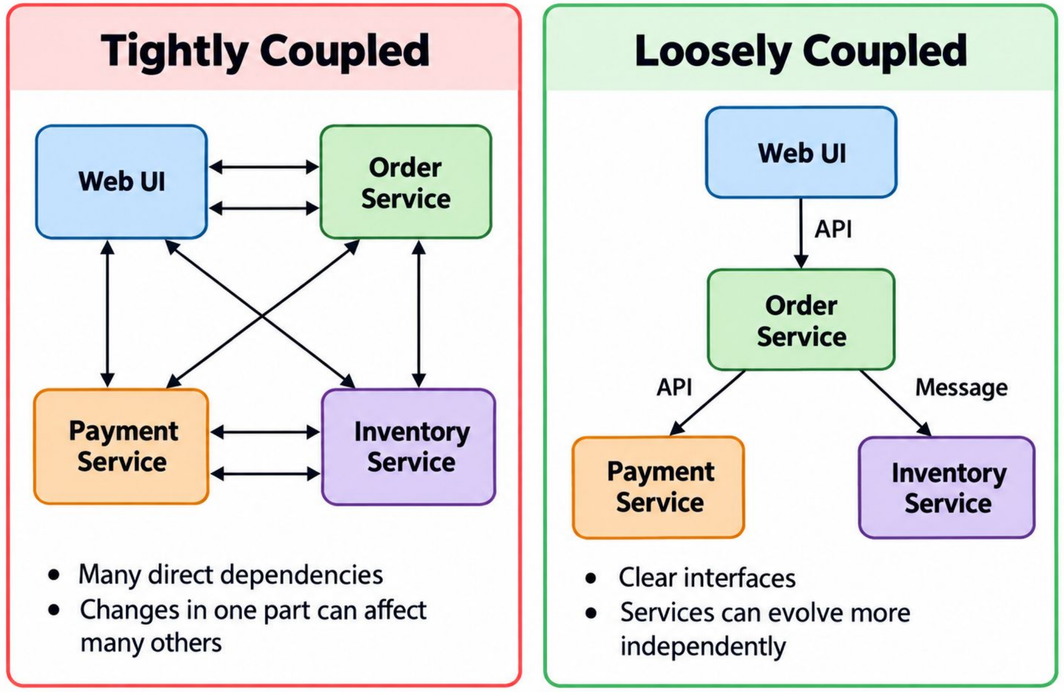
Trade-offs of Microservices

- More distributed-system complexity
- Network latency and communication failures
- More difficult debugging and observability
- More security boundaries
- Data consistency can be harder
- More infrastructure and operational overhead
- Potentially higher cost

Why Decoupling Matters

Decoupling services can be a good idea

- **Tightly coupled:** one component depends heavily on another
- **Loosely coupled:** components interact through clear interfaces with fewer dependencies



Designing Service Boundaries

- Review the application and its responsibilities
- Identify components that could be separated
- Define clear service boundaries
- Define APIs or messages between services
- Decide how data flows between services
- Consider latency, security, failures, and cost

Finding Service Boundaries with Domain-Driven Design

- Start from the business domain, not from technical layers
- Identify major business capabilities or responsibilities
- Group closely related data and behavior together
- Define clear boundaries between responsibilities
- A service should own a focused business capability

Example: Online Store

- Product
- Order
- Payment
- Inventory
- Shipping
- User

Good service boundaries reduce unnecessary dependencies between services.

Domain-Driven Design (DDD)

Best Practice - An Introduction To Domain-Driven Design

Case Study: Simplifying an Architecture

Prime Video case

- Original architecture used many distributed/serverless components
- High-volume communication and intermediate data processing created significant overhead
- The team consolidated some processing into a simpler architecture
- The reported result was substantially lower operational cost

Key lesson: More distributed services are not always better; at scale, coordination and data movement can become a major cost.

Best of 2023: Microservices Sucks — Amazon Goes Back to Basics

No One-Size-Fits-All Architecture

- Do not over-engineer the solution
- Start with application requirements, not technology
- Monoliths have advantages
- Microservices have advantages
- Choose architecture based on scale, complexity, team structure, cost, and expected change

Don’t build to a buzzword.



A Practical Middle Ground

- Start with a simple architecture when appropriate
- Keep stable functionality together
- Extract services when there is a clear reason
- Good candidates may include components that:
  - need independent scaling
  - change frequently
  - have clear business boundaries
  - are owned by different teams

Architecture can evolve over time.

Lab: Monolith to Microservices

In this lab, you will deploy an existing monolithic application to Google Kubernetes Engine (GKE) and then gradually break it into independently deployed microservices. The lab can be found here.

Lab objectives

- Deploy the monolithic application
- Verify that it works
- Extract one component into a microservice
- Deploy the new service independently
- Reconfigure the application to use the new service
- Verify that everything still works
- Repeat for additional components



Migration is incremental — we do not replace the whole monolith at once.

Copyright Google LLC. For educational purposes in accordance with the terms of use set forth on the program Website.

Codelab: Common Issues and Troubleshooting

If npm is not found:

curl -sL https://deb.nodesource.com/setup_20.x | sudo -E bash -

sudo apt install -y nodejs

sudo apt install -y build-essential

curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash

If the monolith service does not deploy correctly:

- Check for command or configuration errors.
- Delete the existing service if necessary:

delete: kubectl delete service monolith

- Run the deployment step again.

Skill Lab: Monolith to Microservices

Google Skills provides another version of the Monolith-to-Microservices exercise.

Skill Lab: *Migrating a Monolithic Website to Microservices on Google Kubernetes Engine*

The lab can be found **here**.



Lab objectives

- Deploying a monolithic application on GKE
- Breaking application components into separate services
- Deploying and updating services independently
- Reconfiguring the application as services are extracted

Example: updating a deployed service (assuming you made frontend:2.0.0):

```
kubectl set image deployment/frontend frontend=gcr.io/${GOOGLE_CLOUD_PROJECT}/frontend:2.0.0
```

Copyright Google LLC. For educational purposes in accordance with the terms of use set forth on the program Website.

Summary

- A **monolith** contains many functions in one main deployment unit.
- **Microservices** divide an application into focused, independently deployable services.
- Microservices can improve independent development, deployment, and scaling.
- Microservices also introduce more networking, operational, security, and debugging complexity.
- Good service boundaries reduce unnecessary dependencies.
- Microservices are **not always better**; architecture should match the application requirements.
- Migration from a monolith to microservices can be **incremental**.

Practice



For each scenario, decide whether you would keep a monolith, move toward microservices, or use a mixed approach. Briefly explain why.

1. A small student project has three developers, a few hundred users, and a simple web application with login, posts, and comments.

2. An e-commerce application has Product, Order, Payment, and Shipping functions. Product traffic grows rapidly during sales, while the other functions remain relatively stable.

3. A large application has several teams. Payment changes frequently and has different security and scaling requirements, while most other functionality is stable.